Welcome to your next lesson: Lists i: creating lists.

I'm preparing your personalized lesson based on what you've already learned, what's next for you, and the best way to help you move forward.

Step by step, you’re moving forward.

Imagine you want to store your scores from 4 games: 10, 15, 8, 20 🎯

One way is:

```
score1 = 10
score2 = 15
score3 = 8
score4 = 20
```

That works, but it’s messy and hard to work with.

A list lets you put them all in one variable, like a row of boxes:
[10, 15, 8, 20]

You’ll learn how to create these in Python and work with them.

To get a feel for where you are:

Have you used lists (arrays) in any language before, or would you say this is brand new?

I've used arrays before

Nice, that helps a lot 👍

In Python, a list is very much like an array in other languages.

First, let’s start simple with literal list creation.

In Python, you create a list with square brackets and commas, like this:

Python

```
scores = [10, 15, 8, 20]
print(scores)
```

This is like saying “one variable, holding 4 values in order.”

Does this basic syntax match what you expected from your array experience, or does anything look odd so far?

It looks familiar

Perfect 👌 That makes this smoother.

Next step: still on creating lists, but I want you to do one.

In Python, here’s the pattern again:

Python

```
favorite_numbers = [3, 7, 21]
print(favorite_numbers)
```
Your turn 👉

Open the code editor.
Create a list named scores with exactly 4 integers of your choice, using square brackets.
Add print(scores) so you can see it.
Run, then Submit code.

Do that, and I'll check it and we'll build from there.

Python

```
scores = [2, 7, 12, 13]
print(scores)
```
Nice, Ron — that's a correct Python list 🎯

You just created a list literal and printed it: scores = [2, 7, 12, 13].

Now, I'll show you another way to create lists: using range().

range(1, 5) means: 1, 2, 3, 4 (stop value not included).

If we wrap it in list(...), we see the numbers:

Python

```
nums = list(range(1, 5))
print(nums)   # shows [1, 2, 3, 4]
```
So: list(range(start, stop)) → list of integers from start up to (but not including) stop.

Does list(range(1, 5)) make sense so far, especially that 5 is not included?

Yes, that makes sense

Great ☑

Let's do a quick hands-on step with range().

Remember:

range(1, 6) → 1, 2, 3, 4, 5
6 is not included, it's the "stop" limit.
Your turn in the code editor:

Create a variable called nums using list(range(3, 10)).
print(nums) to see the list.

So you'll end up with two lines of code.

Run it, then Submit code, and I'll check the result.

Python

```
nums = list(range(3,10))
print(nums)
```
Nice work ☑ nums is exactly [3, 4, 5, 6, 7, 8, 9].

Let's add one more key idea: length of a list.

In Python, len(...) tells you how many items are in a list:

Python

```
nums = [10, 20, 30]
print(len(nums))   # this will print 3
```

So len(nums) is "how many boxes are in this row."

Quick check before we practice:

If a list is [5, 6, 7, 8], what do you expect len([5, 6, 7, 8]) to be? (Just say the number.)

4

Exactly 👍 len([5, 6, 7, 8]) is 4 because there are 4 items.

Now let's have you use len() with a list you create.

In the code editor:

Create a list called scores with 5 numbers (any integers you like) using square brackets.
Print the list.
On the next line, print its length using len(scores).

So you should have something like:

Python

```
scores = [...]
print(scores)
print(len(scores))
```

Fill in your own numbers, run it, then Submit code so I can check it.

Python

```
scores = [5, 6, 7, 12, 13]
print(scores)
print(len(scores))
```

Nice, Ron — that's perfect ☑

You created a 5-item list and len(scores) correctly printed 5.

Next key idea: lists are mutable.

Plain language:

Mutable means you can change something in place, instead of making a new thing.

With a list, you can change one item, and the same list variable now looks different. For example:

Python

```
songs = ["A", "B", "C"]
songs[0] = "Z"
print(songs)   # now it's ["Z", "B", "C"]
```

You didn't make a new list; you edited the first spot.

Does this idea of "change in place" match what you've seen with arrays before?

Yes, it makes sense

What do you think? Pick the option that really changes the list 👇

Which line of Python code changes an existing list in place (shows that it’s mutable)?

A.

```
nums = [1, 2, 3, 4]
```

B.

```
nums = list(range(1, 5))
```

C.

```
nums = [1, 2, 3, 4]
nums[0] = 99
```

D.

```
length = len([1, 2, 3, 4])
```

Nice choice 🎯

`nums = [1, 2, 3, 4]` creates the list, and

`nums[0] = 99` changes the first element in place — same list, new content.

That’s exactly what mutable means in practice.

Now I’d love for you to try a tiny mutation yourself:

In the code editor:

Create a list called breakfast with 3 strings (foods), e.g. ["eggs", "toast", "coffee"].
print(breakfast)
Change the first item, like breakfast[0] = "pancakes".
Print breakfast again to show it changed.

Then Submit code, and I’ll check that your list mutation looks good.

Python

```
breakfast = ["eggs","toast", "coffee"]
print(breakfast)
breakfast[0] = "pancakes"
print(breakfast)
```

Love it 👏

You clearly showed mutability:

First print: ['eggs', 'toast', 'coffee']
After breakfast[0] = "pancakes":
Second print: ['pancakes', 'toast', 'coffee']

Same list variable, different first item → clear “changed in place.”

Last key idea for this lesson: when is a list better than separate variables?

Imagine you did:

Python

```
score1 = 10
score2 = 20
score3 = 15
```

score4 = 18
vs:

Python

scores = [10, 20, 15, 18]
With the list, you have one variable holding all related values, which is easier to pass around, change, and inspect.

Quick check: in a simple game where you track 5 level scores, would you personally prefer:

5 separate variables (score1, score2, …), or
1 list like scores = [...]?

Just answer: “separate variables” or “one list”.

One list

What do you think? Pick the option that gives you a list from 1 through 10 👇

You want a list of the numbers from 1 to 10 (including both 1 and 10). Which code correctly creates it in Python?

A.

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9]
B.

nums = list(range(1, 11))
C.

nums = list(range(1, 10))
D.

nums = len(range(1, 11))
Nice job on that one 🎯

list(range(1, 11)) is correct because:

range(1, 11) gives numbers 1 through 10 (11 is the stop, not included).
Wrapping with list(...) turns that range into an actual list.

So you end up with [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] ☑

You’ve now:

Created lists with literals and with list(range())
Used len() to get list length
Shown that lists are mutable by changing breakfast[0]
Chosen lists over separate variables in a scenario

You’ve hit all the goals for this lesson 💪

Are you ready to wrap up this lesson and move on to the next one about working with lists in more detail? (Yes / No)

Yes, I'm ready

Awesome work today, Ron 🎉

Quick recap of what you can now do:

Create Python lists with square brackets, like scores = [5, 6, 7, 12, 13]

Build lists from ranges with list(range(start, stop)), remembering the stop value isn’t included
Use len() to get how many items are in a list
Show that lists are mutable by changing an element in place, like breakfast[0] = "pancakes"
Recognize when one list is better than a bunch of separate variables
You’re in a good spot to go deeper into lists next 😎
When you’re ready, hit Next Lesson in the Learn tab, and we’ll keep building from here.

How would you rate this lesson?